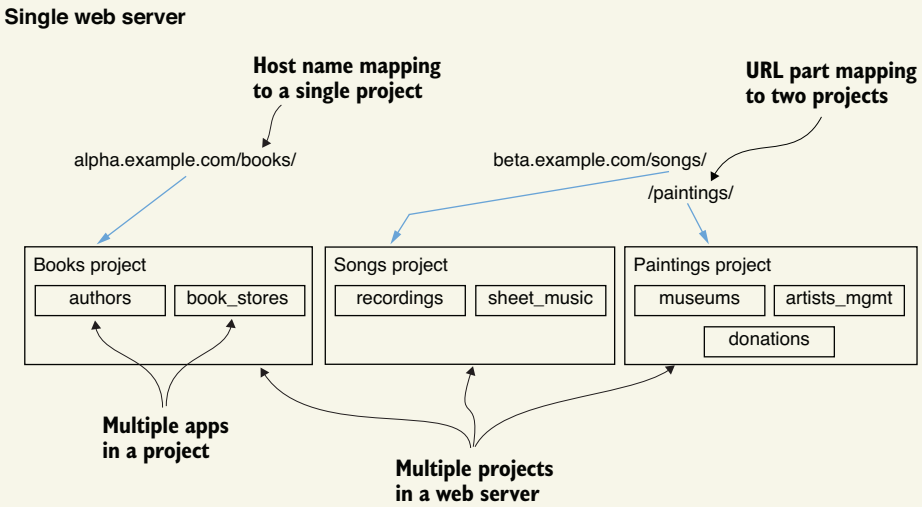
To help organize your code, Django has a concept built on top of Python modules: apps. This term was used by Django before it became the common name of programs on your phone. In Django's terminology, your site is a *project* and your modules are *apps*.

You need at least one app in your project. For smaller projects containing a single app, the name of that app isn't all that important. I often call mine *core*. With larger projects that have multiple apps, you want to name your apps something descriptive of what they do. This is no different from a larger Python program containing multiple module directories.

Apps in projects in web servers

Even small sites can benefit from the organization principle of having several apps within a project. And although the most common configuration is for a single project in a web server, this isn't a requirement. A web server can host multiple Django projects, as long as it is clear which URLs go to which projects. This decision might be based on a host name in web servers configured to support multiple hosts, or based on part of the URL, with the first segment of the URL path pointing to different projects. The figure shows an example of multiple Django projects, each of which has multiple Django apps.



Multiple apps per project, multiple projects per server

You should design your Django apps to be self-contained. The idea of an app even extends to third-party libraries. You can install apps written by other people, and there are thousands of them available to you out there. How you organize your apps and how much they reference each other is up to you, but the structure is in place to help you stay organized.